

Programming for Applications Teaching System: Architecture and Operations Guide

Yu-You Liou

2026-08-02

Contents

1	System Overview	3
1.1	Course Structure	3
1.2	Assessment Structure	3
2	Directory Architecture	4
2.1	Root Directory Structure	4
2.2	Design Principles	5
3	Technology Stack	6
3.1	Core Tools	6
3.2	Required R Packages	6
3.3	Dataset Loading Convention	6
3.4	Website Publication Architecture	7
4	Lecture Slide Authoring Workflow	8
4.1	File Format	8
4.2	Authoring Standards	8
4.3	Generating a New Chapter	9
5	Problem Set Management	10
5.1	Directory Mapping	10
5.2	Problem Set Format	10
5.3	Submission and Grade Distribution Channels	10
5.4	Submission Requirements	10
6	Automated Grading System	11
6.1	Exercise Grading: Process Overview	11
6.2	Scoreboard Format (<code>ex-score.xlsx</code>)	11
6.3	Analytics Report (<code>stat.Rmd</code>)	11
6.4	Grading Note Codes	12
6.5	Examination Grading Pipeline (<code>exam-review.Rmd</code>)	12
6.6	Semester Grade Aggregation (<code>sem-review.Rmd</code>)	13
7	Academic Integrity Mechanisms	14
7.1	UUID Verification	14
7.2	Examination Regulations	14
7.3	AI Use Policy	14
8	AI Prompt Management	15
8.1	Exercise Grading Prompt (<code>ex-review.Rmd</code>)	15
8.2	Examination Grading Prompt (<code>exam-review.Rmd</code>)	15
8.3	Semester Aggregation Prompt (<code>sem-review.Rmd</code>)	15

9 Operations Guide	16
9.1 New Semester Initialization	16
9.2 Weekly Problem Set Publication	16
9.3 Weekly Submission Collection and Grading	16
9.4 Examination Grading	16
9.5 End-of-Semester Grade Aggregation	16
9.6 Generating a New Lecture Chapter	17
9.7 Full-Site Publication (GitHub Pages)	17
9.8 Troubleshooting Reference	17
10 Course Chapter Reference	18
10.1 Weekly Schedule	18
10.2 Chapter Index	18
11 Appendix	20
11.1 A. Official Submission Template	20
11.2 B. Directory Quick Reference	20
11.3 C. References	20

1 System Overview

This document describes the complete architecture and operational procedures of the **Programming for Applications Teaching System (pa)**, designed for the English Taught Program in International Business (ETP) at Shih Chien University. Like the Data Analysis system (**da**), this is a **single-semester course**; the system therefore carries no semester index — there is exactly one roster directory, one submission tree, and one pair of exam directories. The system is built on a core technology stack of R, Quarto, and R Markdown, integrating website publication, in-class exercises, and automated grading into a unified workflow.

Course website: <https://yyliou.github.io/pa> **Instructor:** Yu-You Liou (d10627008@ntu.edu.tw)

1.1 Course Structure

Table 1: Course Identification and Scheduling

Course	Code	Semester	Schedule	Room	Chapters
Programming for Applications	EIB-302-01-A1	Fall 2026	Tuesday, Periods 8–9 (15:10–17:00)	A207	Ch. 1–26

Each session is structured as two consecutive hours: the first hour is a **Lecture** covering foundational knowledge and concepts, and the second hour is a **Lab** in which students apply concepts through in-class exercises using the R programming language.

1.2 Assessment Structure

Table 2: Assessment Weighting and Components

Component	Weight	Details
In-Class Exercises	36%	12 exercises x 3% each
Midterm Examination	32%	
Final Examination	32%	

2 Directory Architecture

2.1 Root Directory Structure

The system root (pa/) is organized as follows:

```
pa/
|-- index.html          # Published homepage (rendered output)
|-- sylla.html          # Published syllabus landing page
|-- slide.html          # Published slide index
|-- prob.html           # Published problem set page
|-- r.html              # Published R guide
|-- site_libs/          # Quarto-generated site assets
|-- hex/                # Course hex logo (pa.svg)
|
|-- web/                # Website page sources (qmd) and configuration
|   |-- web.Rproj        # RStudio project file
|   |-- _quarto.yml      # Quarto website global configuration
|   |-- _brand.yml       # Brand (theme) configuration
|   |-- index.qmd        # Homepage (course overview, exam regulations)
|   |-- sylla.qmd        # Syllabus landing page (links to PDF syllabus)
|   |-- slide.qmd        # Slide index page (Parts I-VI)
|   |-- prob.qmd         # Problem set page (template, submission links)
|   |-- r.qmd            # R installation and usage guide
|
|-- ch1/ ... ch26/      # Per-chapter lecture slide directories
|   |-- chN.qmd          # English Reveal.js slide source
|   |-- chN-zh.qmd       # Traditional-Chinese parallel version
|   |-- bib.bib          # Chapter bibliography
|   |-- logo.svg         # Slide logo asset
|
|-- pa-ps/              # In-class exercise problem sets
|   |-- exN.qmd          # Exercise source (Reveal.js format)
|   |-- exN.html         # Compiled HTML for publication
|
|-- pa-ex/              # Student submissions
|   |-- ex1/ ... ex12/
|       |-- exN-[StudentID].pdf
|
|-- pa-mid/             # Midterm materials and grading outputs
|   |-- mid-score.xlsx   # Exam scoreboard (per-question scores)
|   |-- stat.Rmd         # Analytics report source
|   |-- stat.pdf         # Compiled analytics report
|-- pa-fin/             # Final exam materials and grading outputs
|
|-- stu/                # Roster and grading outputs
|   |-- [roster].xls     # Official roster exported from university system
|   |-- ex[N]/
|       |-- ex-score.xlsx
|       |-- stat.Rmd
|       |-- stat.pdf
|   |-- sem/            # Semester-aggregate outputs
|       |-- sem-score.xlsx
|       |-- stat.Rmd
|       |-- stat.pdf
|
|-- sylla/              # Syllabus source files
|   |-- syllabus.Rmd / syllabus.pdf
|
```

```
|-- !-Archive/          # Archive of retired materials
|
|-- prompt/            # AI-assisted prompt documents
|   |-- prompt.Rmd      # Slide generation prompt
|   |-- ex-review.Rmd   # In-class exercise grading prompt
|   |-- exam-review.Rmd # Midterm/final exam auto-grading prompt
|   |-- sem-review.Rmd  # Semester grade aggregation prompt
|   |-- render.txt      # Batch render command for all chapter slides
|   |-- pa-system-guide.Rmd # This document
```

2.2 Design Principles

The directory structure adheres to six organizational principles:

1. **One directory per chapter.** Directories `ch1/` through `ch26/` are self-contained units (sources, bibliography, logo, rendered output), facilitating independent version control and selective rendering.
2. **Bilingual slide parity.** Every chapter maintains an English source (`chN.qmd`) and a structurally identical Traditional-Chinese version (`chN-zh.qmd`), cross-linked from the title slide.
3. **Separation of page sources and published output.** Website page sources and configuration live in `web/`; rendered HTML is published from the repository root for GitHub Pages.
4. **Separation of problem sets and submissions.** `pa-ps/` (instructor-authored questions) and `pa-ex/` (student submissions) maintain distinct responsibilities and access patterns.
5. **Co-location of roster and grading outputs.** The `stu/` directory holds both the official class roster and all automated grading artifacts (per-exercise and semester-aggregate), enabling single-location lookup.
6. **Centralized prompt management.** All AI-assisted documents are isolated within `prompt/`, decoupled from pedagogical content.

Because the course spans a single semester, no directory carries a semester suffix: the suffix-free names `stu/`, `pa-ex/`, `pa-mid/`, and `pa-fin/` are unambiguous.

Privacy rule. The repository is public (GitHub Pages). The directories `stu/`, `pa-ex/`, `pa-mid/`, and `pa-fin/` contain student personal data (roster, submissions, per-question scores) and are therefore excluded from version control via `.gitignore`, together with all knitr `*_cache/` directories. They exist only in the local (pCloud-synced) working copy and must never be committed or pushed.

3 Technology Stack

3.1 Core Tools

Table 3: Core Software Components

Tool	Minimum Version	Purpose
R	≥ 4.2	Statistical computation and document generation
RStudio Desktop	Latest stable	Integrated development environment
Quarto	≥ 1.3	Website and slide publication
R Markdown	≥ 2.20	Assignment templates and grading reports
TinyTeX	Current	LaTeX engine for PDF output

3.2 Required R Packages

The following packages must be installed prior to system operation:

```
# Document generation
install.packages(c("rmarkdown", "knitr", "tinytex"))
tinytex::install_tinytex()

# Data manipulation and visualization
install.packages(c("tidyverse", "ggplot2", "lattice"))

# Statistics chapters (Part V)
install.packages(c("MASS", "rpart", "nnet", "randomForest", "e1071"))

# Data chapters (Part III)
install.packages(c("DBI", "RSQLite", "readxl"))

# Spreadsheet I/O for grading reports
install.packages(c("readxl", "openxlsx"))

# Academic integrity (UUID generation)
install.packages("uuid")
```

3.3 Dataset Loading Convention

The textbook distributes its example datasets through the companion R packages `nutshell`, `nutshell.bdb`, and `nutshell.audioscrobber`, loaded in the book via `library(nutshell)` and `data(xxx)`. These packages have been archived from CRAN, so **the course materials never use them**. Every lecture slide instead loads datasets directly from the read-only CRAN mirror on GitHub:

```
# Book style (do not use):
# library(nutshell); data(field.goals)
# Course style (download to a temp file, then load;
# do NOT use load(url(...)) - it fails on some platforms):
temp <- tempfile(fileext = ".rda")
download.file("https://raw.githubusercontent.com/cran/nutshell/master/data/field.goals.rda",
             temp, mode = "wb")
load(temp)
```

Available datasets in `cran/nutshell` (directory `data/`): `GSE2034`, `SPECint2006`, `batting.2008`, `births2006.smpl`, `consumption`, `doctorates`, `dow30`, `field.goals`, `ham.price.ts`, `medicare.payments`, `medicare.payments.by.state`, `mort06.smpl`, `outcome.of.care.measures.national`, `sanfrancisco.home.sales`, `shiller`, `spambase`, `team.batting.00to08`, `tires.sus`, `top.bacon.searching.cities`, `toxins.and.cancer`, `turkey.price.ts`, `yosemite`. Datasets from the companion packages follow the same pattern with the repository name `nutshell.bdb` or `nutshell.audioscrobber`.

3.4 Website Publication Architecture

The course website is built as a **Quarto Website** project, governed by `web/_quarto.yml` (with theme branding in `web/_brand.yml`). Key configuration decisions are summarized below.

Table 4: Quarto Website Configuration Summary

Setting	Value
output-dir	.. (renders from web/ directly into the repository root for GitHub Pages; no manual copying)
Page sources	web/ (index, sylla, slide, prob, r)
Theme	Minty + brand (<code>_brand.yml</code>)
Logo / favicon	hex/pa.svg
Navigation bar	Syllabus -> Slides -> Problem sets -> R
Slide format	Reveal.js
Math rendering	KaTeX

4 Lecture Slide Authoring Workflow

4.1 File Format

Each chapter's slides reside in `chN/chN.qmd` (English) and `chN/chN-zh.qmd` (Traditional Chinese), following the official textbook *R in a Nutshell: A Desktop Quick Reference* (2nd ed.). The YAML header is **fixed**; only the subtitle and the language-switch link target may vary:

```
---
title: "Programming for Applications"
subtitle: "Chapter N: [Title]"
author: "Yu-You Liou (NTU)"
institute: "Shih Chien University"
date: "2026-08-02"
format: revealjs
smaller: true
theme: serif
footer: "R in a Nutshell: A Desktop Quick Reference"
logo: "logo.svg"
editor: source
bibliography: bib.bib
suppress-bibliography: true
slide-number: true
scrollable: false
transition: "none"
menu:
  useTextContentForMissingTitles: false
  hideMissingTitles: true
  side: 'left'
  titleSelector: 'h1'
---
```

The header additionally includes an `include-after-body` script and title-slide attributes that inject a language-switch link (`chN-zh.html` in the English file; `chN.html` in the Chinese file). In the Chinese file the title, subtitle, author, and institute fields are localized (`author: " "`, `institute: " "`).

4.2 Authoring Standards

Key conventions:

- Each slide begins with a level-2 heading (`##`); part-level sections use level-1 headings (`#`).
- Mathematical expressions use standard LaTeX syntax, e.g., $\bar{x} = \frac{\sum x}{n}$.
- Definitions and key concepts are enclosed in Quarto callout blocks, which **must be perfectly balanced** (unbalanced callouts are the most common rendering failure):

```
::: {.callout-note}
**Definition** ...
:::
```

- Every textbook example provides an executable R implementation with displayed output. Chunks that install software, modify the system, or require external services use `eval=FALSE`.
- Datasets are always loaded with the GitHub `download.file()` + `load()` convention described above — never `library(nutshell)` and never `load(url(...))`.
- All prose must be original; verbatim reproduction of textbook text is prohibited.
- **Bilingual parity:** `chN.qmd` and `chN-zh.qmd` must be structurally identical, slide for slide.
- **Depth tiering:** Parts I–IV (Ch. 1–15) receive exhaustive coverage; Part V (Ch. 16–23, compressed into four teaching weeks) and Part VI (Ch. 24–26, self-study) may be condensed to core concepts with runnable examples (see `prompt.Rmd`, Instruction 2).
- Existing chapters (`ch1/–ch26/`) serve as the clean reference standard for layout and style.

4.3 Generating a New Chapter

- Step 1. Create the chapter directory with shared assets:
mkdir chN/ and copy bib.bib and logo.svg from an existing chapter.
- Step 2. Provide the slide generation prompt (prompt/prompt.Rmd) to an AI assistant and request chN.qmd and chN-zh.qmd for the target textbook chapter, replicating the structure of an existing chapter directory.
- Step 3. Render both language versions (prompt/render.txt has the batch loop for all chapters):
quarto render chN/chN.qmd
quarto render chN/chN-zh.qmd
- Step 4. Update the chapter index in web/slide.qmd.

5 Problem Set Management

5.1 Directory Mapping

Table 5: Problem Set and Submission Directory Mapping

Course	Problem Set Directory	Submission Directory
Programming for Applications	pa-ps/	pa-ex/

5.2 Problem Set Format

Problem sets are authored as Reveal.js presentations (`pa-ps/exN.qmd`), with one question per slide. A representative header:

```
---
title: "Programming for Applications"
subtitle: "In-Class Exercise N"
author: "Yu-You Liou"
date: today
institute: "Shih Chien University"
format:
  revealjs:
    html-math-method: katex
---
```

5.3 Submission and Grade Distribution Channels

Submission and grade lookup are handled through shared **pCloud** links published on the problem set page (`web/prob.qmd`): one upload link (students place the PDF in the matching `exN` folder, which syncs to `pa-ex/exN/`), and one read-only link for checking grades.

5.4 Submission Requirements

Students must use the official template below and upload to the course pCloud folder `exN/` (synced to `pa-ex/exN/`) using the exact naming convention `exN-[StudentID].pdf`.

```
---
title: "Ex. N"
author: "StudentID"
date: "2026-08-02"
output: pdf_document
---
```

```
\` \` \`{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
\` \` \`

\` \` \`{r}
print(uuid::UUIDgenerate())
\` \` \`
```

Every submission must include the UUID generation code block. This serves as the primary mechanism for academic integrity verification.

6 Automated Grading System

The system comprises three AI-driven grading pipelines: (1) in-class exercise grading (`ex-review.Rmd`), operating on PDF submissions; (2) examination grading (`exam-review.Rmd`), operating on Google Form responses; and (3) semester aggregation (`sem-review.Rmd`), which consolidates all components into the final grade.

6.1 Exercise Grading: Process Overview

The grading pipeline consists of four sequential stages:

```
STAGE 1 | Initialization and Roster Reconciliation
  Read the roster in stu/ -> construct student roster
  Cross-reference pa-ex/exN/ -> flag absent students
  as Missing (Score = 0)

STAGE 2 | Compliance Filtering (sequential; stops at first violation)
  (i) Submission timestamp > [deadline] (UTC+8)
      -> Late Submission (Score = 0)
  (ii) File is not PDF or does not match exN-[StudentID].pdf
      -> Incorrect File Name/Format (Score = 0)
  (iii) YAML missing title/author/date, or UUID chunk absent,
        or author field does not match submitting student ID
      -> Incorrect Internal Format (Score = 0)
  (iv) UUID value identical to another submission, or
        invalid/forged UUID
      -> Plagiarism (Score = 0; all implicated students)

STAGE 3 | Academic Content Grading
  Total: 100 points, allocated equally across N questions
  Each question scored against pa-ps/exN.qmd answer key
  Incorrect items noted as: [QN] is incorrect

STAGE 4 | Output Generation
  stu/exN/ex-score.xlsx <- Scoreboard (4 columns)
  stu/exN/stat.Rmd      <- Analytics report source
  stu/exN/stat.pdf      <- Compiled analytics report
```

6.2 Scoreboard Format (`ex-score.xlsx`)

Table 6: Scoreboard Column Specification

Column	Type	Description
StudentID	Character	University student identification number
Name	Character	Student full name as listed in official roster
Score	Integer (0–100)	Final score awarded for the submission
Note	Character	Compliance violation code or incorrect question numbers

6.3 Analytics Report (`stat.Rmd`)

The analytics report reads `ex-score.xlsx` and produces three outputs:

1. **Descriptive statistics table:** minimum, Q_1 , median, Q_3 , maximum, and mean of the score distribution.
2. **Boxplot:** a publication-quality visualization of the score distribution rendered via `ggplot2`.
3. **Submission summary table:** counts for Total, Missing, Late Submission, Incorrect Format, and Plagiarism.

6.4 Grading Note Codes

Table 7: Grading Note Code Reference

Code	Description	Score
Missing	No file submitted by the deadline	0
Late Submission	File submitted after the official deadline	0
Incorrect File Name/Format	File is not PDF or does not follow naming convention exN-[ID].pdf	0
Incorrect Internal Format	YAML header incomplete, author ID mismatch, or UUID chunk absent	0
Plagiarism	UUID value matches another submission, or is invalid/forged	0
[QN] is incorrect	Question N answer is incorrect or methodology is flawed	Partial deduction

6.5 Examination Grading Pipeline (`exam-review.Rmd`)

Examinations are submitted online through a Google Form whose responses populate a Google Sheet with the fixed schema:

```
time | ip | user | id | name | q1 | q1ans | q2 | q2ans | ... | qN | qNans
```

where `qk` holds the student's R code for Question k and `qkans` its reported output. The pipeline takes three parameters — `[sheet]` (the Google Sheet URL), `[questions]` (the official question set with answers), and `[exam]` (mid or fin) — plus the campus IP range constant `[campus-ip]`.

```
STAGE 1 | Data Acquisition and Roster Reconciliation
  Download [sheet] as CSV; infer N from column pairs;
  keep only the latest row per student ID.
  Roster students with no submission -> Missing (Score = 0)

STAGE 2 | Integrity Filtering (sequential; stops at first violation)
  (i) ip outside [campus-ip]
      -> Invalid IP (off-campus) (Score = 0)
  (ii) user value shared across different student IDs
      -> Duplicate device (Score = 0; all implicated students)

STAGE 3 | Per-Question Grading with Partial Credit
  Total: 100 points, allocated equally across N questions.
  Each question's score is recorded individually.
  Partial-credit rubric:
    Full -> correct method and matching answer
    2/3 -> correct method/code, minor slip in reported answer
    1/3 -> correct method, mis-transcribed input values
    0 -> wrong method, irrelevant code, or blank

STAGE 4 | Output Generation ([dir] = pa-mid/ or pa-fin/)
  [dir]/[exam]-score.xlsx <- Scoreboard (ID, Name, Q1..QN,
    Score, Note)
  [dir]/stat.Rmd <- Analytics report source
  [dir]/stat.pdf <- Compiled analytics report
```

Compared to exercise grading, the exam analytics report adds a **per-question average-score bar chart**, and the submissions summary counts Invalid IP and Duplicate Device cases instead of Late/Plagiarism.

6.6 Semester Grade Aggregation (`sem-review.Rmd`)

Run once, at the end of the semester, after all components are graded. Parameters: the component weights `[w-mid]`, `[w-fin]`, `[w-ex]` (must sum to 100; current syllabus values 32/32/36). There is no semester parameter — the course is single-semester.

```
INPUT    pa-mid/mid-score.xlsx          (midterm)
         pa-fin/fin-score.xlsx          (final)
         stu/ex1..ex12/ex-score.xlsx    (12 exercises)

COMPUTE  ExAvg = mean(Ex1..Ex12)
         Total = w-mid/100 * Mid + w-fin/100 * Fin + w-ex/100 * ExAvg
         (rounded to 2 decimals)

OUTPUT   stu/sem/sem-score.xlsx
         Columns: StudentID, Name, Ex1..Ex12, ExAvg, Mid, Fin,
         Total, Note
         stu/sem/stat.Rmd + stat.pdf
         Descriptive statistics of Total; boxplot; component-mean
         bar chart; histogram (bin width 10); pass/fail summary
         (Total >= 60); weights recap
```

Robustness rules: the roster is the master index (no student silently dropped); a missing component file or absent student contributes 0 and is flagged in `Note`; zero-score trigger reasons are carried over in compact form (e.g., `Mid: Invalid IP`); 3 randomly sampled Totals are re-computed by hand for verification.

7 Academic Integrity Mechanisms

7.1 UUID Verification

Every submission must execute and display output from the following R code block:

```
print(uuid::UUIDgenerate())
```

`uuid::UUIDgenerate()` produces a Version 4 UUID (Universally Unique Identifier) on each invocation, e.g., `6d2b9fda-2d9b-48f3-84bc-f82eabc39b4d`. The grading system compares this value across all submissions for a given exercise. If two or more files share an identical UUID, or if a file's UUID is invalid or forged, all implicated students receive a score of zero with the note **Plagiarism**.

Important implementation notes:

- The compliance filter is applied sequentially per file. A file that fails Stage 2 check (iii) (Incorrect Internal Format) is not evaluated for plagiarism within its own record; however, its UUID remains in the comparison pool and may trigger a Plagiarism flag in other students' records.
- The `author` field in the YAML header must contain the submitting student's own ID. Use of another student's ID constitutes a format violation independent of the plagiarism check.

7.2 Examination Regulations

In alignment with the course's paperless policy, examinations are conducted online. Examination materials are maintained in `pa-mid/` and `pa-fin/`.

Table 8: Examination Conduct Regulations

Regulation	Policy
Mandatory attendance	Roll call 5 minutes before start; late arrival or absence results in score = 0
Network restriction	System tracks IP addresses; campus Wi-Fi required; non-campus IP results in score = 0
Authorized resources	Open-book; reference materials and AI tools are permitted as auxiliary aids
Media prohibition	Photography, video/audio recording, and screenshotting result in score = 0
Communication prohibition	Interpersonal communication and disruptive behavior result in score = 0
Copy-paste disabled	Copy-paste functionality is disabled by the examination system

7.3 AI Use Policy

Students may use AI tools (e.g., ChatGPT or Gemini) as learning aids. However, students must be capable of explaining and defending any code or analysis they submit. Uncritical submission of AI-generated content without demonstrated understanding constitutes academic dishonesty and is subject to the same consequences as plagiarism.

8 AI Prompt Management

The `prompt/` directory contains structured prompt documents for AI-assisted course material generation and grading.

Table 9: Prompt Directory File Inventory

File	Purpose
<code>prompt.Rmd</code>	Slide generation prompt (includes the dataset-loading convention)
<code>ex-review.Rmd</code>	Automated grading prompt for in-class exercises
<code>exam-review.Rmd</code>	Midterm/final examination auto-grading prompt (Google Form pipeline)
<code>sem-review.Rmd</code>	Semester grade aggregation prompt (weighted total of all components)
<code>render.txt</code>	Batch quarto render command for all chapter slides
<code>pa-system-guide.Rmd</code>	This technical document

8.1 Exercise Grading Prompt (`ex-review.Rmd`)

Key components of the grading prompt:

- Definition of environment variables: `[deadline]` (UTC+8), `[no]` (exercise number), and root directory structure (`stu/`, `pa-ps/`, `pa-ex/`).
- Specification of all four grading stages with precise conditional logic.
- Output format requirements: scoreboard column names, Rmd content sections, and PDF compilation.

8.2 Examination Grading Prompt (`exam-review.Rmd`)

Parameters: `[sheet]` (Google Sheet URL of form responses), `[questions]` (official question set with answers), `[exam]` (`mid/fin`), and the constant `[campus-ip]`. Implements the four-stage examination pipeline described in the Automated Grading System section: CSV acquisition and roster reconciliation, integrity filtering (campus IP, duplicate device), per-question grading with the full/2-3/1-3/0 partial-credit rubric, and output to `pa-mid/` or `pa-fin/`.

8.3 Semester Aggregation Prompt (`sem-review.Rmd`)

Parameters: weights `[w-mid]`, `[w-fin]`, `[w-ex]` (sum = 100; no semester parameter). Consolidates the midterm, final, and 12 exercise scoreboards into `stu/sem/sem-score.xlsx` with the weighted Total, carries over zero-score reasons into the Note column, and produces the semester analytics report (`stat.Rmd/stat.pdf`).

9 Operations Guide

9.1 New Semester Initialization

1. Export official class roster from the university administrative system. Save to `stu/`.
2. Update `sylla/syllabus.Rmd` with the current semester's schedule and policies. Knit to PDF.
3. Verify `web/_quarto.yml` navigation links and semester metadata in `web/index.qmd` and `web/sylla.qmd`.
4. Create or empty the pCloud submission folders (`ex1-ex12`) and confirm the upload/grade-lookup links in `web/prob.qmd`.
5. Rebuild the website pages and republish.

9.2 Weekly Problem Set Publication

1. Create `pa-ps/exN.qmd` in Reveal.js format. Each slide corresponds to one question.
2. Render the problem set:
`quarto render pa-ps/exN.qmd`
3. Update the problem set index in `web/prob.qmd`.

9.3 Weekly Submission Collection and Grading

1. Students upload `exN-[StudentID].pdf` to the course pCloud folder, which syncs to `pa-ex/exN/`.
2. Execute the AI grading prompt (`prompt/ex-review.Rmd`) with `[no]` set to the exercise number and `[deadline]` set to the submission deadline (UTC+8).
3. Verify generated scoreboard: `stu/exN/ex-score.xlsx`
4. Verify analytics report: `stu/exN/stat.pdf`
5. Grades are published to students via the read-only pCloud link.

9.4 Examination Grading

1. Conduct the exam online; responses collect in the Google Sheet.
2. Execute the AI exam grading prompt (`prompt/exam-review.Rmd`) with `[sheet]` set to the response sheet URL, `[questions]` set to the official question set (with answers), and `[exam]` = mid or fin.
3. Verify scoreboard: `pa-mid/mid-score.xlsx` (or `pa-fin/fin-score.xlsx`)
- per-question scores plus zero-score reasons.
4. Verify analytics report: `stat.pdf` in the same directory.

9.5 End-of-Semester Grade Aggregation

1. Confirm all 12 exercise scoreboards and both exam scoreboards exist.

2. Execute the AI aggregation prompt (prompt/sem-review.Rmd) with the syllabus weights ([w-mid]/[w-fin]/[w-ex] = 32/32/36).
3. Verify stu/sem/sem-score.xlsx (row count = roster size) and the semester analytics report stat.pdf.
4. Submit final grades to the university system.

9.6 Generating a New Lecture Chapter

1. Create chN/ and copy bib.bib and logo.svg from an existing chapter.
2. Generate chN.qmd and chN-zh.qmd with an AI assistant, replicating the structure and style of the existing chapters (see prompt/prompt.Rmd, including the GitHub dataset-loading rule).
3. Review, edit, and finalize both files in RStudio.
4. Render (batch loop in prompt/render.txt):
for f in ch*/ch*.qmd; do quarto render "\$f"; done
5. Add the chapter link to web/slide.qmd.

9.7 Full-Site Publication (GitHub Pages)

```
# Render updated pages/slides, e.g.
for f in ch*/ch*.qmd; do quarto render "$f"; done
# Website pages render from web/ straight into the repo root:
( cd web && quarto render )

# Commit and push to trigger GitHub Pages deployment
git add .
git commit -m "Update: Week N - [brief description]"
git push origin main
```

9.8 Troubleshooting Reference

Table 10: Common Issues and Resolutions

Symptom	Probable Cause	Resolution
PDF output fails	TinyTeX not installed or incomplete	Run: tinytex::install_tinytex()
UUID package not found	uuid package not installed	Run: install.packages('uuid')
Quarto render error	Quarto version outdated	Update Quarto to latest stable release
Dataset URL fails to load	No internet access or GitHub unreachable	Check connection; datasets stream from raw.githubusercontent.com
Roster file read error	Legacy .xls format compatibility	Use readxl package or resave as .xlsx
Slide math not rendering	KaTeX configuration missing	Set html-math-method: katex in YAML

10 Course Chapter Reference

The course follows the official textbook *R in a Nutshell: A Desktop Quick Reference* (2nd ed.), organized into six parts across 26 chapters.

10.1 Weekly Schedule

Table 11: Weekly Schedule and Exercise Mapping

Week	Part / Chapters	Topic	Exercises
1	Part I (Ch. 1–4)	R basics: installation, user interface, tutorial, packages	ex1
2–4	Part II (Ch. 5–10)	The R language: overview, syntax, objects, environments, functions, OOP	ex2–ex4
5–6	Part III (Ch. 11–12)	Working with data: saving, loading, editing, preparing	ex5–ex6
7	Part IV (Ch. 13)	Base graphics	ex7
8–9	—	Midterm presentations / Midterm Examination	—
10	Part IV (Ch. 14–15)	Lattice graphics; <code>ggplot2</code>	ex8
11–14	Part V (Ch. 16–23)	Statistics: analyzing data, distributions, tests, power, regression, classification, machine learning, time series	ex9–ex12
15–16	—	Final presentations / Final Examination	—
17–18	—	Flexible Learning Weeks; Part VI (Ch. 24–26) self-study	—

10.2 Chapter Index

Table 12: Chapter Index (mirrors web/slide.qmd)

Part	Chapter	Title
I	Ch. 1	Getting and Installing R
I	Ch. 2	The R User Interface
I	Ch. 3	A Short R Tutorial
I	Ch. 4	R Packages
II	Ch. 5	An Overview of the R Language
II	Ch. 6	R Syntax
II	Ch. 7	R Objects
II	Ch. 8	Symbols and Environments
II	Ch. 9	Functions
II	Ch. 10	Object-Oriented Programming
III	Ch. 11	Saving, Loading, and Editing Data
III	Ch. 12	Preparing Data
IV	Ch. 13	Graphics
IV	Ch. 14	Lattice Graphics
IV	Ch. 15	<code>ggplot2</code>
V	Ch. 16	Analyzing Data
V	Ch. 17	Probability Distributions
V	Ch. 18	Statistical Tests
V	Ch. 19	Power Tests
V	Ch. 20	Regression Models

Part	Chapter	Title
V	Ch. 21	Classification Models
V	Ch. 22	Machine Learning
V	Ch. 23	Time Series Analysis
VI	Ch. 24	Optimizing R Programs
VI	Ch. 25	Bioconductor
VI	Ch. 26	R and Hadoop

11 Appendix

11.1 A. Official Submission Template

```

---
title: "Ex. 1"
author: "A113290001"
date: "2026-09-22"
output: pdf_document
---

# This chunk is mandatory in every submission.
print(uuid::UUIDgenerate())

# Exercise solutions follow below.

```

11.2 B. Directory Quick Reference

Table 13: Directory Purpose Quick Reference

Directory	Purpose
web/	Website page sources, Quarto/brand configuration, RStudio project
ch1/-ch26/	Bilingual Reveal.js lecture slide sources, one subdirectory per chapter
pa-ps/	In-class exercise problem sets
pa-ex/exN/	Student PDF submissions for exercise N
pa-mid/, pa-fin/	Exam materials and grading outputs ([exam]-score.xlsx, stat.pdf)
stu/exN/	Exercise grading outputs (scoreboard, analytics report)
stu/sem/	Semester-aggregate scoreboard and analytics report
sylla/	Syllabus R Markdown source files
hex/	Course hex logo (pa.svg)
prompt/	AI prompts and this technical document
!-Archive/	Archive of retired materials

11.3 C. References

- Adler, J. *R in a Nutshell: A Desktop Quick Reference* (2nd ed.). O'Reilly Media.
- Wickham, H., Navarro, D., & Pedersen, T. L. *ggplot2: Elegant Graphics for Data Analysis* (3rd ed.). <https://ggplot2-book.org>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org>
- Posit, PBC. (2024). *RStudio Desktop*. <https://posit.co/download/rstudio-desktop>
- Allaire, J. J., et al. (2024). *Quarto*. <https://quarto.org>
- CRAN GitHub mirror of the nutshell data packages: <https://github.com/cran/nutshell>
- Course website: <https://yyliou.github.io/pa>